Automating Net Banking Flows Using Generative AI

Abstract

This article presents a case study on integrating generative AI into an existing Maven BDD automation framework for net banking applications. By leveraging Claude for intelligent prompting and self-healing, the project demonstrates how AI can accelerate test case generation, improve maintainability, and reduce manual effort. The approach is illustrated through a basic login and logoff flow, with emphasis on data preparation, framework integration, and benefits achieved.

Introduction

Automation frameworks are widely used to streamline repetitive testing tasks in banking applications. Traditional frameworks, while robust, often require significant manual effort in test case creation and maintenance. Generative AI offers a new paradigm by enabling intelligent automation, adaptive test generation, and self-healing capabilities. This study explores the integration of Claude into a Maven BDD framework (Java 1.8, Selenium) to automate net banking flows across multiple entities (countries). The focus is on the login and logoff scenario, which serves as a foundation for more complex flows.

Methodology

Framework Setup

- **Base Framework**: Maven BDD automation framework (existing setup reused).
- **Technology Stack**: Java 1.8, Selenium WebDriver.
- **AI Component**: Claude for intelligent prompting and self-healing.

Scenario Outline

The baseline flow automated was the **Login → Dashboard → Logoff** sequence:

1. Open browser.
2. Navigate to net banking URL → Login page appears.
3. Enter username → Click *Continue*.
4. Enter password → Click *Logon*.
5. Dashboard loads → Wait until fully rendered.
6. Click *Logoff*.

Data Preparation

- **XPath Identification**: Located XPath for all UI elements (username textbox, continue button, password field, logon button, dashboard, logoff button).
- **Excel Repository**: Created an Excel file containing:
    - UI elements (buttons, links, textboxes)
    - Corresponding XPath values
    - Additional comments/metadata

AI Integration

- The Excel file was provided to Claude.
- Claude generated:
    - **BDD Test Cases** in *Given–When–Then* format
    - **Step Definition Files** automatically
- Credentials (username, password) were externalized into a .properties file for security and maintainability.

Results

- **End-to-End Execution**: Test cases executed successfully without manual intervention.
- **Reduced Effort**: Automated generation of test cases and step definitions minimized repetitive coding.
- **Self-Healing**: AI prompts adapted to minor UI changes, reducing test failures.
- **Scalability**: Framework can extend to other flows (fund transfer, account summary, etc.) across multiple entities.
- **Maintainability**: Centralized Excel repository and property files simplified updates.

Example (BDD Format)

gherkin

Scenario: Net Banking Login and Logoff

Given the browser is opened

When the user navigates to the login page

And enters the username

And clicks continue

And enters the password

And clicks logon

Then the dashboard should be displayed

And the user logs off successfully

Conclusion

This case study demonstrates that integrating generative AI into existing automation frameworks can significantly enhance efficiency, adaptability, and scalability. By automating test case generation and enabling self-healing, AI reduces manual effort and improves reliability. The approach is particularly valuable for complex, multi-entity applications such as net banking, where frequent UI changes and diverse workflows demand robust automation.